



SORBONNE UNIVERSITÉ
Campus Sciences et Ingénierie, Jussieu

**COMPTE RENDU PROJET : SRE et
observabilité.**

KITOKO DAVID 28706212

KANDIL OMAR : 21107819

CRV

Introduction

Dans le cadre de ce projet, nous avons mis en place une infrastructure containerisée orchestrée avec Kubernetes, intégrant des outils de monitoring comme Prometheus et Grafana. L'objectif est d'observer le comportement de l'application sous différentes charges, en étudiant l'impact de ces variations sur les performances du serveur et de la base Redis.

Pour simuler l'activité des utilisateurs et générer de la charge, un script `scenario.sh` a été développé. Celui-ci lance différentes requêtes HTTP sur les endpoints du serveur, en alternant lectures (GET /, GET /item, GET /unsafe) et écritures (POST /item). Plusieurs scénarios ont été testés : charge progressive, surcharge rapide et montée en charge constante.

Grâce à Grafana, nous avons pu visualiser et analyser les métriques récupérées par Prometheus. Les graphiques suivants illustrent les effets de ces scénarios sur l'activité des endpoints, l'utilisation des ressources et le comportement global du système.

Etudes de quelques scénarios



FIGURE 1 – Lors de l'observation des métriques via Grafana, on remarque que l'application supporte un trafic conséquent avec un nombre important de requêtes HTTP réussies et une forte sollicitation de la base Redis, qui reste performante sous la charge. Cependant, des points d'attention apparaissent : un volume élevé d'erreurs 500 indique que le serveur rencontre des difficultés à traiter toutes les requêtes, probablement en raison d'une saturation ou d'un défaut applicatif. On note également un nombre non négligeable d'erreurs 404, signe de requêtes vers des ressources inexistantes. Ces résultats soulignent l'intérêt de mettre en place une montée en charge automatique des pods et d'améliorer la gestion des erreurs pour garantir la stabilité du système sous forte charge.

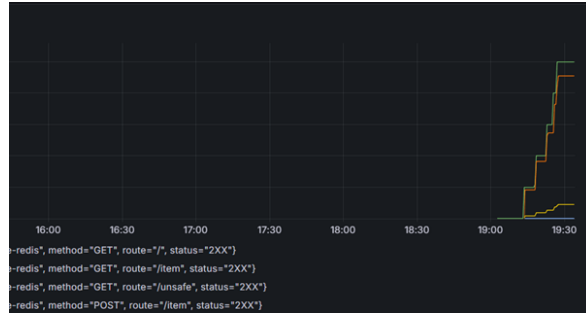


FIGURE 2 – Ce graphique montre l'évolution du nombre de requêtes HTTP réussies (codes 2XX) envoyées vers différents endpoints de l'application. On observe une montée progressive et simultanée des requêtes sur les routes principales (/, /item, /unsafe), ce qui correspond à la montée en charge générée par le scénario de test. La synchronisation de ces courbes indique que les différents endpoints sont sollicités de façon équilibrée, ce qui permet de tester l'application dans des conditions réalistes et variées. Cette tendance confirme que l'infrastructure encaisse correctement la montée du trafic sans rupture immédiate de service.



FIGURE 3 – Ce graphique synthétise l'état des différentes métriques de l'application sous charge, en affichant des seuils colorés pour chaque endpoint et la consommation CPU. Les indicateurs passent au rouge lorsque les valeurs dépassent les seuils critiques (80+), et restent en vert lorsqu'elles sont en-dessous. On remarque ici que la majorité des routes GET et POST atteignent des niveaux élevés d'activité (en rouge) à partir de 19h, signe d'une montée en charge importante et rapide, tandis que la consommation CPU reste maîtrisée (en vert). Ce contraste montre que malgré un trafic soutenu, le serveur conserve une bonne stabilité sur le plan des ressources processeur.

Conclusion

Cette expérimentation a permis de mieux comprendre le comportement de l'application face à différentes charges simulées par `scenario.sh`. Grâce au monitoring en temps réel avec Grafana et Prometheus, nous avons observé l'impact de la charge sur les performances du serveur et de Redis, ainsi que la montée en charge et les limites du système. Ces observations confirment l'importance du dimensionnement des ressources et du scaling automatique pour garantir la stabilité et la réactivité d'une application en production.